



ChargeSync

Intelligent EV Charging Reservation and Recommendation Platform

Software Requirements Specification

SE3090 – Software Engineering Frameworks | Assignment 1

BSc (Hons) in Information Technology, SLIIT — Year 3, Semester 1, 2026

Document Version 2.0

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Scope	4
1.3 Definitions, Acronyms and Abbreviations	4
1.4 Document Overview	5
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Functions	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints	6
3. System Features	7
3.1 Function 1 — Vehicle & AI Compatibility Discovery (Student 1).....	7
CRUD Operations.....	7
Functional Requirements	7
API Endpoints.....	8
3.2 Function 2 — Station, Charger & Operating-Hours Management (Student 2).....	8
CRUD Operations.....	9
Functional Requirements	9
API Endpoints.....	9
3.3 Function 3 — Reservation & AI Charging Planning (Student 3).....	10
CRUD Operations.....	10
Functional Requirements	10
API Endpoints.....	11
3.4 Function 4 — Session, Payment, Loyalty & Support Management (Student 4).....	12
CRUD Operations.....	12
Functional Requirements	12
API Endpoints.....	13
4. Data Requirements (Database Design)	14
4.1 Entity Relationship Overview.....	14
4.2 Data Dictionary	14

Users (shared)	14
ConnectorTypes (Function 1)	15
Vehicles (Function 1).....	15
VehicleChargingProfiles (Function 1).....	16
Stations (Function 2).....	16
Chargers (Function 2)	17
OperatingHours (Function 2).....	18
MaintenanceWindows (Function 2).....	18
Reservations (Function 3)	19
ReservationStatusHistory (Function 3).....	20
WaitlistEntries (Function 3).....	20
ChargingPlans (Function 3)	21
ChargingSessions (Function 4).....	21
PaymentInvoices (Function 4)	22
MembershipPlans (Function 4).....	23
Subscriptions (Function 4).....	24
LoyaltyAccounts (Function 4)	24
RewardRedemptions (Function 4)	25
SupportTickets (Function 4)	25
AgentWorkflowRuns (shared, Agentic AI)	26
5. External Interface Requirements.....	27
5.1 User Interfaces	27
5.2 API Interfaces	28
5.3 Software Interfaces	28
5.4 Third-Party Interfaces	28
6. System Architecture and Component Interconnections.....	28
6.1 High-Level Architecture	28
6.2 Operational Workflows.....	28
Advance Reservation Flow	28
Unregistered Walk-In Flow	29
Session Termination & Settlement Flow	29
7. Non-Functional Requirements	29

7.1 Performance	29
7.2 Security & Fraud Prevention	29
7.3 Reliability and Availability	30
7.4 Quality Assurance and Continuous Integration	30
8. Agentic AI Subsystem Requirements	30
8.1 Agent Overview & Endpoints.....	30
Agentic AI Endpoints	30
8.2 Human-in-the-Loop Approval Triggers.....	31
9. Appendices.....	32
9.1 Summary of Architecture Decision Records	32
9.2 Revision History	33

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for ChargeSync, an integrated full-stack and Agentic AI application developed for the SE3090 - Software Engineering Frameworks Assignment 1. This Version 2.0 incorporates comprehensive adaptations for emerging markets characterized by varied smartphone digital literacy, lack of direct vehicle-to-charger IoT telemetry, and high prevalence of cash transactions through a hybrid driver/staff point-of-sale (POS) operational workflow.

1.2 Document Scope

This document covers the four core business components (Functions 1–4), their CRUD operations, complete relational database schema across 19 entities, external interface requirements, cross-component data flow, non-functional requirements, and the Agentic AI subsystem with deterministic human-in-the-loop triggers.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
FCM	Firebase Cloud Messaging
ORM	Object-Relational Mapping (Entity Framework Core)
ADR	Architecture Decision Record
LLM	Large Language Model
POS	Point of Sale (Staff mobile and web operational workflow)
UUID	Universally Unique Identifier (RFC 4122 v4)
3NF	Third Normal Form (database normalization standard)
CDN	Content Delivery Network (Cloudinary media asset delivery)
HMAC	Hash-based Message Authentication Code (Internal service key authentication)
GiST	Generalized Search Tree (PostgreSQL temporal range indexing)

1.4 Document Overview

Section 2 provides the overall product description and revised user roles. Section 3 outlines detailed functional requirements, CRUD matrices, and endpoints across all 4 functions. Section 4 specifies the complete revised database schema. Section 5 details interface requirements. Section 6 defines system architecture and operational cross-function flows. Section 7 presents non-functional requirements. Section 8 details the Agentic AI subsystem. Section 9 provides the revised Architecture Decision Records (ADRs).

2. Overall Description

2.1 Product Perspective

ChargeSync is a self-contained system comprising a shared ASP.NET Core Web API and PostgreSQL database, a React web portal for station owners, platform administrators, and support staff, a Flutter mobile application serving both EV drivers and on-site station staff, and an internal Python/FastAPI/LangGraph Agentic AI service. The Agentic AI service is reachable strictly via the ASP.NET Core backend and is never exposed directly to client applications.

2.2 Product Functions

- **Function 1 (Student 1):** Vehicle registration, driver vehicle management, platform administrator vehicle management across all drivers, and AI-driven vehicle-charger compatibility calculation.
- **Function 2 (Student 2):** Station, charger, operating-hours, and maintenance-window administration with real-time availability reconciliation.
- **Function 3 (Student 3):** Conflict-free reservation booking secured by advance virtual wallet payment, unique reservation QR token generation, staff-operated QR check-in, manual walk-in allocation, and AI charging planning.
- **Function 4 (Student 4):** Staff-monitored charging session lifecycle tracking, hybrid baseline energy calculation with staff physical meter override and Cloudinary meter photo verification, automated invoice generation supporting cash and wallet settlement, membership subscriptions, loyalty tracking, and support ticket triage.
- **Agentic AI Subsystem:** A four-agent Agentic AI subsystem orchestrating multi-agent state machines with deterministic human-in-the-loop review triggers.

2.3 User Classes and Characteristics

Role	Description
EV Driver	Mobile app user. Registers vehicles, requests AI charging plans, books slots via advance wallet payment, displays reservation QR codes, tracks charging history, and accesses loyalty rewards.

Role	Description
Station Staff / Station Owner	Operates the React portal for administrative station setup, pricing, charger provisioning, and utilization analytics (general support ticket triage is restricted to Platform Administrators and Support Staff). Operates the Flutter mobile app on-site to scan driver booking QR codes, admit unregistered walk-in drivers, inspect physical charger meters, capture meter photo evidence, enter energy overrides, stop sessions, and log cash payments.
Platform Administrator	Full administrative access. Manages all registered driver vehicles across the platform (create, view, update, delete any vehicle), reviews support tickets and meter photo verification records, approves stations, audits discrepancies between mathematical energy calculations and staff manual overrides, reviews AI-flagged actions, and manages user accounts.
Customer Support Manager	Manages support tickets, refund requests, and billing disputes; coordinates administrative overrides with station operators.

2.4 Operating Environment

- **Backend:** ASP.NET Core Web API (C# .NET 8), deployed on cloud infrastructure.
- **Database:** PostgreSQL 16 with btree_gist extension enabled for temporal exclusion locking.
- **Web Client:** React Single Page Application (SPA) with TanStack Query and Tailwind CSS.
- **Mobile Client:** Flutter application targeting Android (APK), providing dual role-based dashboards (Driver and Station Staff POS).
- **Agentic AI Service:** Python 3.11 + FastAPI + LangGraph, containerized as an internal microservice.
- **Media Storage:** Cloudinary CDN for station images and physical checkout meter reading photos.
- **CI/CD Pipeline:** GitHub Actions pipelines for backend, web, and mobile targeting both main and dev branches.

2.5 Design and Implementation Constraints

- All client communications must route exclusively through the ASP.NET Core API.
- The Agentic AI service must remain an internal service invoked solely by ASP.NET Core, authenticated via an internal service authentication key (X-Agent-Service-Key).
- No external payment gateway integration is required; payments are managed via an internal wallet ledger and staff cash logs.
- No IoT/hardware telemetry is required; session energy is derived via automated mathematical calculation and reconciled against manual staff meter input and photographic evidence.

3. System Features

3.1 Function 1 — Vehicle & AI Compatibility Discovery (Student 1)

Allows drivers to register and manage their vehicles, discover nearby stations, and receive an AI-computed compatibility score before travelling to a charger, preventing wasted journeys to incompatible facilities. In v2.0, administrative vehicle oversight is expanded to allow Platform Administrators full platform-wide vehicle management.

CRUD Operations

Operation	Description
Create	Register vehicle (make, model, connector type, battery capacity, max charge rate) by driver or platform administrator.
Read	View registered vehicles (drivers view their own; administrators view all platform vehicles or filter by driver), search stations, view compatibility scores.
Update	Edit vehicle specifications (drivers update their own; administrators update any vehicle across all drivers).
Delete	Remove vehicle (drivers delete their own; administrators delete any vehicle record without ownership restriction).

Functional Requirements

Req. ID	Requirement Description	Priority
FR-1.1	Driver shall register a vehicle specifying make, model, connector type, battery capacity, and max charge rate.	High
FR-1.2	Driver shall view, update, and delete their own vehicles only.	High
FR-1.3	System shall compute a compatibility score and estimated charging time for a vehicle/charger pair.	High
FR-1.4	System shall suggest alternative stations if incompatibility is detected.	Medium
FR-1.5	Driver shall search nearby stations filtered by connector type and radius.	High
FR-1.6	Platform Administrator shall view all registered vehicles across all drivers system-wide, search by driver, make, model, or license plate, filter by connector type, and create, update, or delete any vehicle record.	High

Req. ID	Requirement Description	Priority
FR-1.7	System shall allow authorized administrative roles (Admin, StationOwner) to retrieve vehicles belonging to a specific user.	Medium

API Endpoints

Method	Endpoint	Purpose
POST	/api/vehicles	Register a new vehicle
GET	/api/vehicles	List current driver's registered vehicles
GET	/api/vehicles/all	List all registered vehicles across all drivers (Admin only)
GET	/api/vehicles/user/{userId}	Retrieve all vehicles for a specified user (Admin, StationOwner)
GET	/api/vehicles/{id}	Retrieve specific vehicle details
PUT	/api/vehicles/{id}	Update own vehicle specifications
PUT	/api/vehicles/admin/{id}	Update any vehicle specifications without ownership restriction (Admin only)
DELETE	/api/vehicles/{id}	Delete own vehicle record
DELETE	/api/vehicles/admin/{id}	Delete any vehicle record without ownership restriction (Admin only)
GET	/api/vehicles/{id}/compatible-stations	Find compatible charging stations for a specific vehicle
GET	/api/stations/nearby	Search nearby stations by coordinates and connector type
GET	/api/stations/{id}/compatibility	Get compatibility score and charge time estimate

3.2 Function 2 — Station, Charger & Operating-Hours Management (Student 2)

Allows station owners to manage physical charging infrastructure, configure operating schedules, schedule maintenance downtime, and monitor live facility utilization.

CRUD Operations

Operation	Description
Create	Register station, add charger, configure maintenance windows and operating hours schedule.
Read	View owned stations, charger statuses, operating hours, real-time availability, analytics.
Update	Update station information, charger rates, operating hours, operational state.
Delete	Remove charger or station (subject to validation ensuring no active bookings exist).

Functional Requirements

Req. ID	Requirement Description	Priority
FR-2.1	Station owner shall register a station with location coordinates, address, and operating details.	High
FR-2.2	Station owner shall add, update, and remove chargers under their owned stations.	High
FR-2.3	Station owner shall configure weekly operating hours per day of week.	High
FR-2.4	Station owner shall schedule charger maintenance windows.	Medium
FR-2.5	System shall compute real-time availability by reconciling operating hours, maintenance windows, active reservations, and walk-in locks.	High
FR-2.6	System shall provide utilization and revenue analytics per station.	Medium

API Endpoints

Method	Endpoint	Purpose
POST	/api/stations	Register a new station
GET	/api/stations	List, filter, and search stations
GET	/api/stations/{id}	Retrieve station details and chargers
PUT	/api/stations/{id}	Update station profile and location

Method	Endpoint	Purpose
DELETE	/api/stations/{id}	Remove station (if no active bookings)
POST	/api/stations/{id}/chargers	Add charger to station
PUT	/api/chargers/{id}	Update charger specifications and pricing
PUT	/api/chargers/{id}/status	Update operational status (Available/Occupied/Maintenance)
GET	/api/stations/{id}/utilization	Retrieve station utilization and revenue analytics

3.3 Function 3 — Reservation & AI Charging Planning (Student 3)

Coordinates AI-driven route and schedule recommendations, prevents double-booking using PostgreSQL GiST exclusion locks, manages waitlists, generates advance reservation QR tokens, and provides staff-assisted check-ins and walk-in slot allocations.

CRUD Operations

Operation	Description
Create	Create a reservation with advance virtual wallet payment; generate AI plan; admit walk-in customer; join waitlist.
Read	View reservations, generated QR code, waitlist position, and full status history.
Update	Modify reservation time window (subject to slot availability reconciliation).
Delete	Cancel reservation (triggers wallet refund policy and automated waitlist promotion).

Functional Requirements

Req. ID	Requirement Description	Priority
FR-3.1	Driver shall create a reservation for an available charger slot by submitting an advance deposit via their virtual wallet.	High
FR-3.2	System shall generate a cryptographically signed, unique ReservationQRCode token upon successful advance payment deduction.	High

Req. ID	Requirement Description	Priority
FR-3.3	System shall prevent overlapping bookings using PostgreSQL GiST exclusion constraints and pessimistic row locking.	High
FR-3.4	Station Staff shall scan the driver's presented QR code via the Flutter mobile app to validate identity and execute check-in.	High
FR-3.5	Station Staff shall admit unregistered walk-in drivers by creating an on-demand reservation record with a null DriverId, locking the charger for immediate use.	High
FR-3.6	System shall maintain a waitlist per charger and automatically promote the next queue entry upon reservation cancellation.	Medium
FR-3.7	System shall generate an AI-ranked charging plan given driver constraints (deadline, distance, price preference).	High
FR-3.8	System shall record a full status history for every reservation transition.	Medium
FR-3.9	System shall display physical meter photo evidence (captured upon session completion) within reservation details to enable staff and platform administrators to visually audit meter readings.	High

API Endpoints

Method	Endpoint	Purpose
POST	/api/reservations	Create advance reservation with wallet pre-authorization
POST	/api/reservations/walk-in	Staff-initiated reservation and immediate lock for walk-ins
GET	/api/reservations	List, filter, and paginate reservations
GET	/api/reservations/{id}	Retrieve reservation details, QR payload, and linked session details (including meter photo URL)
PUT	/api/reservations/{id}/cancel	Cancel reservation and trigger waitlist promotion
POST	/api/reservations/staff-checkin	Staff-scanned QR check-in endpoint

Method	Endpoint	Purpose
POST	/api/charging-plan/generate	Generate multi-agent AI charging plan
GET	/api/reservations/{id}/history	Retrieve reservation audit trail

3.4 Function 4 — Session, Payment, Loyalty & Support Management (Student 4)

Handles session monitoring, hybrid mathematical-to-meter energy reconciliation, point-of-sale invoicing, cash/wallet tracking, loyalty programs, and support ticketing.

CRUD Operations

Operation	Description
Create	Initiate charging session; issue invoice; create subscription; submit ticket; log points redemption.
Read	View active session status, historical invoices, staff override logs, loyalty balances, support tickets.
Update	Conclude charging session with physical meter input and photo upload; change subscription plan; update ticket status (Admin/Support only).
Delete	Cancel subscription; withdraw pending support ticket.

Functional Requirements

Req. ID	Requirement Description	Priority
FR-4.1	System shall record session start upon staff QR verification or walk-in admission.	High
FR-4.2	Station Staff shall conclude a charging session via mobile app when charging finishes or when customer unplugs early.	High
FR-4.3	System shall calculate a baseline AutoCalculatedKwh using the formula: $\text{Charger Output (kW)} \times \text{Duration (Hours)}$.	High
FR-4.4	Station Staff shall have the capability to input StaffOverriddenKwh read directly from the physical charger display and attach photographic evidence of the meter display (uploaded to Cloudinary) to substantiate the reading.	High
FR-4.5	System shall compute the final invoice based on StaffOverriddenKwh (if present) or AutoCalculatedKwh, deducting pre-paid reservation deposits.	High
FR-4.6	System shall record settlement method as either 'Wallet' or 'Cash' (collected on-site by staff).	High

Req. ID	Requirement Description	Priority
FR-4.7	System shall flag discrepancies exceeding 15% between auto-calculated and staff-overridden kWh for Platform Administrator fraud review.	High
FR-4.8	System shall track loyalty points for registered users and compute tier status.	Medium
FR-4.9	Driver shall redeem points for rewards subject to validation rules.	High
FR-4.10	System shall triage support tickets and mandate human-in-the-loop review on refund requests exceeding \$15.00 or loyalty redemptions exceeding 5,000 points.	High
FR-4.11	Support ticket management interface shall be restricted to Platform Administrator and Support Staff roles within the web portal; station owners do not have access to general driver support tickets.	High

API Endpoints

Method	Endpoint	Purpose
POST	/api/sessions/start	Record session initiation upon check-in
PUT	/api/sessions/{id}/stop	Stop session, submit physical meter reading, and optional Cloudinary meter photo URL
GET	/api/payments/invoices/{userId}	Retrieve customer invoice ledger
POST	/api/payments/invoices/{id}/settle	Settle invoice via cash logging or wallet charge
POST	/api/subscriptions	Enroll in membership tier
PUT	/api/subscriptions/{id}/change	Update membership tier
GET	/api/loyalty/{userId}	Fetch loyalty balance and tier
POST	/api/loyalty/redeem	Redeem loyalty points for vouchers
POST	/api/support-tickets	Submit driver support ticket
PUT	/api/support-tickets/{id}/status	Update support ticket state (Admin/Support only)

4. Data Requirements (Database Design)

4.1 Entity Relationship Overview

The database is normalized to Third Normal Form (3NF) and optimized for transactional consistency and concurrent access using PostgreSQL 16. Key structural relationships include:

- Users (1) to (N) Vehicles, Stations (as Owner), Reservations, Subscriptions, SupportTickets.
- Vehicles (N) to (1) ConnectorTypes; Vehicles (1) to (1) VehicleChargingProfiles.
- Stations (1) to (N) Chargers, OperatingHours.
- Chargers (1) to (N) MaintenanceWindows, Reservations, WaitlistEntries.
- Reservations (1) to (N) ReservationStatusHistory; Reservations (1) to (1) ChargingSessions.
- Reservations.DriverId is nullable to facilitate unregistered walk-ins without violating foreign key constraints.
- ChargingSessions (1) to (1) PaymentInvoices; ChargingSessions stores MeterPhotoUrl pointing to Cloudinary-stored physical meter photos.
- Users (1) to (1) LoyaltyAccounts; LoyaltyAccounts (1) to (N) RewardRedemptions.
- AgentWorkflowRuns tracks multi-agent planning state and links to ChargingPlans.

4.2 Data Dictionary

Users (shared)

Stores all system accounts across all roles.

Column	Data Type	Constraints	Description
Id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
FullName	VARCHAR(150))	NOT NULL	User's full legal/display name
Email	VARCHAR(150))	NOT NULL, UNIQUE	User login email address
PasswordHash	VARCHAR(255))	NOT NULL	Bcrypt password hash
Role	VARCHAR(30)	NOT NULL, CHECK IN ('Driver', 'StationOwner', 'StationStaff', 'Admin', 'SupportManager')	System role authorization tier
PhoneNumber	VARCHAR(20)	NULL	Contact telephone number
WalletBalance	DECIMAL(10,2))	NOT NULL, DEFAULT 0.00	Internal prepaid wallet balance

Column	Data Type	Constraints	Description
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active/suspended status flag
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Record modification timestamp

ConnectorTypes (Function 1)

Lookup table of supported EV connector standards.

Column	Data Type	Constraints	Description
Id	UUID	PK	Connector standard identifier
Name	VARCHAR(50)	NOT NULL, UNIQUE	Standard name (e.g., CCS2, Type 2, CHAdeMO)
Description	TEXT	NULL	Standard technical specifications and pinout info
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

Vehicles (Function 1)

Driver-owned vehicle records used for compatibility scoring.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique vehicle identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Owning registered driver
Make	VARCHAR(50)	NOT NULL	Vehicle manufacturer (e.g., Hyundai, BYD, Nissan)
Model	VARCHAR(50)	NOT NULL	Vehicle model name (e.g., Ioniq 5, Atto 3, Leaf)
Year	SMALLINT	NULL	Manufacturing year
ConnectorTypeId	UUID	NOT NULL, FK -> ConnectorTypes(Id)	Hardware socket standard

Column	Data Type	Constraints	Description
BatteryCapacityKwh	DECIMAL(6,2)	NOT NULL	Total usable battery storage capacity
MaxChargeRateKw	DECIMAL(6,2)	NOT NULL	Peak DC/AC charging intake limit
LicensePlate	VARCHAR(20)	NULL	Registration vehicle license plate
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

VehicleChargingProfiles (Function 1)

Extended charging preferences/profile data used by the Compatibility Agent.

Column	Data Type	Constraints	Description
Id	UUID	PK	Profile record identifier
VehicleId	UUID	NOT NULL, UNIQUE, FK -> Vehicles(Id)	Linked vehicle record
OptimalChargeRateKw	DECIMAL(6,2)	NULL	Recommended steady-state charge rate
PreferredConnectorTypeId	UUID	NULL, FK -> ConnectorTypes(Id)	Preferred adapter standard
NotesJson	JSONB	NULL	Manufacturer charging curve profiles and thresholds
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

Stations (Function 2)

Charging stations registered by station owners.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique station identifier

Column	Data Type	Constraints	Description
OwnerId	UUID	NOT NULL, FK -> Users(Id)	Owning station operator
Name	VARCHAR(150)	NOT NULL	Station commercial display name
AddressLine	VARCHAR(255)	NOT NULL	Physical street/road address
Latitude	DECIMAL(9,6)	NOT NULL	Geospatial coordinate latitude
Longitude	DECIMAL(9,6)	NOT NULL	Geospatial coordinate longitude
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending', 'Active', 'Suspended')	Operational status tier
CreatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Modification timestamp

Chargers (Function 2)

Individual charging units belonging to a station.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique charger unit identifier
StationId	UUID	NOT NULL, FK -> Stations(Id)	Parent station facility
ConnectorTypeId	UUID	NOT NULL, FK -> ConnectorTypes(Id)	Physical plug configuration
MaxOutputKw	DECIMAL(6,2)	NOT NULL	Rated continuous power capacity
PricePerKwh	DECIMAL(6,2)	NOT NULL	Unit electricity tariff rate
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Available', CHECK IN ('Available', 'Occupied', 'Maintenance')	Operational state

Column	Data Type	Constraints	Description
CreatedAt	TIMESTAMPT Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPT Z	NOT NULL, DEFAULT now()	Modification timestamp

OperatingHours (Function 2)

Weekly operating schedule per station.

Column	Data Type	Constraints	Description
Id	UUID	PK	Schedule entry identifier
StationId	UUID	NOT NULL, FK -> Stations(Id)	Parent station reference
DayOfWeek	SMALLINT	NOT NULL, CHECK (DayOfWeek BETWEEN 0 AND 6)	Day index: 0 = Sunday, 6 = Saturday
OpenTime	TIME	NOT NULL	Opening operational time
CloseTime	TIME	NOT NULL	Closing operational time
CreatedAt	TIMESTAMPT Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPT Z	NOT NULL, DEFAULT now()	Modification timestamp

MaintenanceWindows (Function 2)

Scheduled downtime periods for a specific charger.

Column	Data Type	Constraints	Description
Id	UUID	PK	Maintenance record identifier
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Targeted physical charger unit
StartTime	TIMESTAMPT Z	NOT NULL	Scheduled maintenance start
EndTime	TIMESTAMPT Z	NOT NULL	Scheduled maintenance conclusion
Reason	TEXT	NULL	Maintenance explanation or routine service description

Column	Data Type	Constraints	Description
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Modification timestamp

Reservations (Function 3)

Driver bookings for a charger slot. Protected against double-booking via PostgreSQL exclusion constraint on (ChargerId, tsrange(StartTime, EndTime)).

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique reservation identifier
DriverId	UUID	NULL, FK -> Users(Id)	Registered driver; NULL indicates an on-site walk-in customer
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Targeted physical charger unit
VehicleId	UUID	NULL, FK -> Vehicles(Id)	Registered vehicle profile (optional for walk-ins)
StartTime	TIMESTAMPTZ	NOT NULL	Scheduled slot start timestamp
EndTime	TIMESTAMPTZ	NOT NULL	Scheduled slot end timestamp
ReservationQRCode	VARCHAR(255)	NULL, UNIQUE	Signed QR token for driver presentation and staff scanning
AdvanceDepositAmount	DECIMAL(8,2)	NOT NULL, DEFAULT 0.00	Deposit deducted from virtual wallet on booking
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending', 'Confirmed', 'CheckedIn', 'Cancelled', 'Completed')	Current lifecycle status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation timestamp

Column	Data Type	Constraints	Description
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Modification timestamp

ReservationStatusHistory (Function 3)

Full audit trail of every reservation status change.

Column	Data Type	Constraints	Description
Id	UUID	PK	History record identifier
ReservationId	UUID	NOT NULL, FK -> Reservations(Id)	Parent reservation record
OldStatus	VARCHAR(20)	NULL	Prior status string
NewStatus	VARCHAR(20)	NOT NULL	Transitioned status string
ChangedByUserId	UUID	NULL, FK -> Users(Id)	User or staff identity initiating transition
ChangedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Transition timestamp

WaitlistEntries (Function 3)

Queue of drivers waiting for a fully booked charger slot.

Column	Data Type	Constraints	Description
Id	UUID	PK	Waitlist entry identifier
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Desired charger unit
DriverId	UUID	NOT NULL, FK -> Users(Id)	Queued driver reference
RequestedStartTime	TIMESTAMPTZ	NOT NULL	Requested slot start time
Priority	INT	NOT NULL, DEFAULT 0	Queue priority ranking index
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Waiting', CHECK IN ('Waiting', 'Promoted', 'Expired')	Waitlist state

Column	Data Type	Constraints	Description
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

ChargingPlans (Function 3)

AI-generated itinerary options produced by the Planning Agent for a given driver objective.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique plan identifier
WorkflowRunId	UUID	NULL, FK -> AgentWorkflowRuns(W orkflowRunId)	Originating agent workflow run
DriverId	UUID	NOT NULL, FK -> Users(Id)	Requesting driver
Deadline	TIMESTAMP Z	NOT NULL	Arrival deadline specified by driver
MaxDistanceKm	DECIMAL(6,2)	NULL	Maximum acceptable driving radius
PricePreference	VARCHAR(20)	NULL	Budget/speed sensitivity preference
PlanJson	JSONB	NOT NULL	Ranked itinerary payload produced by AI Coordinator
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

ChargingSessions (Function 4)

Software-tracked charging session lifecycle. Reconciles mathematical estimation with staff physical meter overrides and photo evidence.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique session identifier
ReservationId	UUID	NOT NULL, UNIQUE, FK -> Reservations(Id)	Originating reservation record

Column	Data Type	Constraints	Description
StartTime	TIMESTAMPTZ	NOT NULL	Physical session start (Staff QR check-in timestamp)
EndTime	TIMESTAMPTZ	NULL	Physical session termination timestamp
AutoCalculatedKwh	DECIMAL(8,2)	NULL	System calculated: Charger Output (kW) × Duration (h)
StaffOverriddenKwh	DECIMAL(8,2)	NULL	Manual staff reading from physical charger screen
FinalEnergyDeliveredKwh	DECIMAL(8,2)	NULL	Billed energy value used for invoice settlement
StaffUserId	UUID	NULL, FK -> Users(Id)	Staff member supervising and closing session
MeterPhotoUrl	VARCHAR(500)	NULL	Secure URL to physical meter photo stored on Cloudinary
Status	VARCHAR(20)	NOT NULL, DEFAULT 'InProgress', CHECK IN ('InProgress', 'Completed', 'DiscrepancyFlagged')	Session operational status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Modification timestamp

PaymentInvoices (Function 4)

Internal invoice ledger record, auto-generated on session completion (no external payment gateway).

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique invoice identifier
SessionId	UUID	NOT NULL, UNIQUE, FK -> ChargingSessions(Id)	Originating session reference
DriverId	UUID	NULL, FK -> Users(Id)	Billed driver (NULL for cash-paying walk-ins)

Column	Data Type	Constraints	Description
GrossAmount	DECIMAL(10,2)	NOT NULL	Total cost based on final billed kWh
AdvanceDeducted	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Deposit credited from advance booking
NetAmountDue	DECIMAL(10,2)	NOT NULL	Outstanding settlement balance
PaymentMethod	VARCHAR(20)	NOT NULL, CHECK IN ('Wallet', 'Cash')	Final payment settlement method
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending', 'Paid', 'Refunded')	Invoice settlement status
IssuedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Issuance timestamp
CreatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Modification timestamp

MembershipPlans (Function 4)

Catalog of available subscription tiers.

Column	Data Type	Constraints	Description
Id	UUID	PK	Plan tier identifier
Name	VARCHAR(50)	NOT NULL	Plan display name (e.g., Gold Fleet, Daily Commuter)
Description	TEXT	NULL	Plan privileges and terms
MonthlyFee	DECIMAL(8,2)	NOT NULL	Monthly subscription recurring tariff
DiscountPercentage	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Per-session tariff discount percentage
CreatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Modification timestamp

Subscriptions (Function 4)

A driver's active or historical membership plan enrollment.

Column	Data Type	Constraints	Description
Id	UUID	PK	Subscription identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Subscribed driver user
PlanId	UUID	NOT NULL, FK -> MembershipPlans(Id)	Subscribed membership tier
StartDate	DATE	NOT NULL	Enrollment start date
EndDate	DATE	NULL	Termination/renewal date
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Active', CHECK IN ('Active', 'Cancelled', 'Expired')	Subscription status
CreatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Modification timestamp

LoyaltyAccounts (Function 4)

One-per-driver loyalty points balance and tier.

Column	Data Type	Constraints	Description
Id	UUID	PK	Loyalty account identifier
DriverId	UUID	NOT NULL, UNIQUE, FK -> Users(Id)	Associated driver account
PointsBalance	INT	NOT NULL, DEFAULT 0	Accumulated loyalty points balance
Tier	VARCHAR(20)	NOT NULL, DEFAULT 'Bronze', CHECK IN ('Bronze', 'Silver', 'Gold')	Driver loyalty tier status
CreatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMPZ	NOT NULL, DEFAULT now()	Modification timestamp

RewardRedemptions (Function 4)

Records of points redeemed for a reward; high-value redemptions require Admin approval.

Column	Data Type	Constraints	Description
Id	UUID	PK	Redemption record identifier
LoyaltyAccountId	UUID	NOT NULL, FK -> LoyaltyAccounts(Id)	Redeeming loyalty account
PointsRedeemed	INT	NOT NULL	Points debited for redemption
RewardDescription	VARCHAR(255)	NOT NULL	Reward item description or coupon summary
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending', 'Approved', 'Rejected')	Approval state (AI or Admin gated)
CreatedAt	TIMESTAMP TZ	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP TZ	NOT NULL, DEFAULT now()	Modification timestamp

SupportTickets (Function 4)

Driver-submitted support requests, triaged by the Validation & Support Agent.

Column	Data Type	Constraints	Description
Id	UUID	PK	Ticket identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Submitting driver user
Category	VARCHAR(50)	NOT NULL	Inquiry classification (Billing, Charger, Session)
Priority	VARCHAR(20)	NOT NULL, DEFAULT 'Medium', CHECK IN ('Low', 'Medium', 'High', 'Urgent')	AI-triaged priority score
Description	TEXT	NOT NULL	Driver's detailed problem narrative

Column	Data Type	Constraints	Description
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Open', CHECK IN ('Open', 'InProgress', 'Resolved', 'Closed')	Operational ticket status
AssignedToUserId	UUID	NULL, FK -> Users(Id)	Assigned support staff or administrator
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT now()	Modification timestamp

AgentWorkflowRuns (shared, Agentic AI)

Central audit table persisting the full state of every Agentic AI workflow execution.

Column	Data Type	Constraints	Description
WorkflowRunId	UUID	PK, DEFAULT gen_random_uuid()	Unique execution workflow run identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Requesting driver
Objective	TEXT	NOT NULL	Driver input prompt and constraints
Status	VARCHAR(32)	NOT NULL	State: Running, PendingApproval, Completed, Failed
Plan.Json	JSONB	NULL	Generated structured multi-step plan payload
AgentOutputs.Json	JSONB	NULL	Diagnostic step outputs from sub-agents
ValidationResult.Json	JSONB	NULL	Validation agent audit and compliance report
RequiresApproval	BOOLEAN	NOT NULL, DEFAULT FALSE	Human-in-the-loop review trigger flag
ApprovalDecision	VARCHAR(16)	NULL	Approved, Rejected, or Revised

Column	Data Type	Constraints	Description
ApprovedByUserId	UUID	NULL, FK -> Users(Id)	Reviewing staff or administrator
ApprovalNotes	TEXT	NULL	Reviewer's audit and justification notes
FailureReason	TEXT	NULL	Safe-failure exception detail, if encountered
RetryCount	INT	NOT NULL, DEFAULT 0	Automated recovery retry counter
CreatedAt	TIMESTAMP Z	NOT NULL, DEFAULT clock_timestamp()	Execution start timestamp
UpdatedAt	TIMESTAMP Z	NOT NULL, DEFAULT clock_timestamp()	Modification timestamp

5. External Interface Requirements

5.1 User Interfaces

- **React Web Portal:** Role-based access for Station Owners, Platform Administrators, and Support Staff. Platform Administrators benefit from a dedicated system-wide vehicle management module (/vehicles) featuring a responsive card grid, cross-driver search, connector filter, battery statistics, and full vehicle CRUD modals; station approval workflows; discrepancy fraud audit logs; user management; and support ticket triage. Station Owners manage station registration, charger provisioning, operating hours, maintenance windows, and utilization analytics (general support ticket triage is restricted to Platform Administrators and Support Staff). Reservation Details modals render comprehensive booking telemetry, status timelines, and uploaded Cloudinary physical meter photos for visual inspection.
- **Flutter Mobile Application:** Dual-mode Flutter application targeting Android. Driver Mode provides vehicle profile management, AI charging itinerary generation, interactive station map discovery, advance wallet reservation booking, unique QR token display, charging session history, wallet balance management, and support ticket submission. Staff POS Mode delivers on-site operations: camera-based QR scanner for rapid driver check-in, one-tap walk-in charger locking (accommodating non-app drivers), active charging dashboard, stop-session trigger with physical meter kWh entry, direct Cloudinary meter photo capture/upload, and on-site cash collection settlement logging.

5.2 API Interfaces

All client-server communications use RESTful JSON over HTTPS, secured via JWT bearer tokens and strict role-based access control (RBAC). Swagger UI documentation is automatically generated at /swagger to facilitate API exploration and testing.

5.3 Software Interfaces

- **PostgreSQL Database:** Interfaced via Entity Framework Core with the Npgsql data provider, utilizing PostgreSQL connection pooling and transactional isolation.
- **Agentic AI Service:** Internal Python/FastAPI service called exclusively by the ASP.NET Core AgentClient, secured by an internal service key header (X-Agent-Service-Key) using constant-time string digest comparison.

5.4 Third-Party Interfaces

- **Google Maps API:** Google Maps Places and Distance Matrix APIs for geospatial station discovery, radius filtering, and travel duration calculation.
- **Firebase Cloud Messaging (FCM):** Push notification service to dispatch slot confirmation tokens, waitlist queue promotions, and session completion receipts.
- **Cloudinary API:** High-availability media storage and CDN delivery for charging station banners and checkout physical meter display photo evidence.

6. System Architecture and Component Interconnections

6.1 High-Level Architecture

ChargeSync adopts an enterprise-grade, tiered client-server architecture. Flutter mobile applications (Driver / Staff POS) and the React administrative portal communicate strictly with the ASP.NET Core Web API over TLS 1.3 HTTPS. The API coordinates relational persistence in PostgreSQL and initiates asynchronous or synchronous multi-agent reasoning workflows via the containerized Python/LangGraph AI service.

6.2 Operational Workflows

Advance Reservation Flow

- Driver submits desired charging objective (vehicle, charger, time window) via Flutter app.
- ASP.NET Core verifies slot availability, deducts advance deposit from driver's WalletBalance, inserts Reservations record with GiST exclusion constraints, and returns a signed ReservationQRCode token.
- On arrival at the station, driver presents the generated QR code token.
- Station Staff scans QR code via Flutter POS app; system validates reservation, starts ChargingSessions record, and updates status to CheckedIn.

Unregistered Walk-In Flow

- Driver arrives on-site without an account, mobile application, or advance booking.
- Station Staff checks charger availability on-site. In the Flutter app, staff taps 'Admit Walk-In' for that specific charger.
- ASP.NET Core inserts a Reservations record with DriverId = NULL to lock the charger and prevent online double-bookings, immediately initiating a ChargingSessions record.

Session Termination & Settlement Flow

- When charging concludes or driver unplugs early, Station Staff taps 'Stop Session' in the Flutter POS app.
- System logs EndTime and calculates baseline AutoCalculatedKwh using charger rated output × elapsed hours.
- Staff enters StaffOverriddenKwh read directly from physical charger screen and captures/uploads a photo of the meter display directly to Cloudinary (stored as MeterPhotoUrl) for audit proof.
- System generates PaymentInvoices, crediting pre-paid reservation deposit where applicable.
- Customer settles net balance via virtual wallet or cash paid to staff; staff marks invoice 'Paid'. Discrepancies exceeding 15% between calculated and overridden kWh are flagged for Platform Administrator fraud review.

7. Non-Functional Requirements

7.1 Performance

- Standard CRUD API endpoints shall respond within 500ms at the 95th percentile under expected peak load.
- Agentic AI route planning executions shall complete within 10 seconds under standard conditions.

7.2 Security & Fraud Prevention

- JWT bearer authentication across all mobile and web API requests with strict role-based access control.
- Entity Framework Core parameterized queries to prevent SQL injection vulnerabilities.
- Passwords hashed using industry-standard bcrypt with high work factor.
- **Internal Service Authentication:** Inter-service communication between ASP.NET Core and the Python Agentic AI service is secured via an X-Agent-Service-Key header validated using constant-time string comparison (secrets.compare_digest) to prevent timing attacks.
- **Staff Audit Trail & Photographic Evidence:** Any manual override of energy consumption (StaffOverriddenKwh) permanently records the StaffUserId alongside the AutoCalculatedKwh and Cloudinary MeterPhotoUrl, eliminating cash fraud and substantiating billing.

7.3 Reliability and Availability

- ACID transactions across all multi-step booking, session termination, and invoice settlement operations.
- Third-party service degradations (Maps, FCM) fail gracefully without blocking on-site physical charging.

7.4 Quality Assurance and Continuous Integration

- **CI/CD Automation:** GitHub Actions pipelines configured for push and pull request triggers against both main and dev branches across backend, web-react, and mobile-flutter.
- **Backend Test Suite:** 158+ unit tests implemented with xUnit and Moq covering ReservationService, slot exclusion validation, waitlist transitions, and charging session lifecycles.
- **Agentic AI Test Suite:** 24 pytest unit and integration tests verifying PlanningCoordinatorAgent, compatibility scoring algorithms, and SupportAgent decision matrices.
- **Mobile Test Suite:** Unit and widget tests for PlanningApiClient, SmartRecommendationCard, and AiPlanningScreen utilizing mockable HTTP clients and safe GPS service wrappers.
- **Web Test Suite:** Vitest and React Testing Library tests verifying ReservationDetailsModal, vehicle management CRUD forms, and administrative route guards.

8. Agentic AI Subsystem Requirements

8.1 Agent Overview & Endpoints

The subsystem employs four specialized autonomous agents orchestrated through LangGraph state graphs:

- **Vehicle Compatibility Agent:** Assesses charger-to-vehicle hardware compatibility and calculates charging time.
- **Station Analysis Agent:** Evaluates real-time charger status, historical pricing, and utilization scores.
- **Charging Recommendation & Planning Agent (Coordinator):** Synthesizes driver objectives, coordinates Compatibility and Station agents, and generates ranked itineraries. Must handle records with null DriverId without failing availability models.
- **Validation & Support Agent:** Inspects session records, checks business constraints, flags overrides, and handles support triage.

Agentic AI Endpoints

Method	Endpoint	Purpose
POST	/api/compatibility/evaluate	Single vehicle/charger hardware compatibility assessment

Method	Endpoint	Purpose
POST	/api/compatibility/batch-evaluate	Batch compatibility screening across multiple candidate stations
POST	/api/station-analysis/evaluate	Station utilization analysis and dynamic pricing assessment
POST	/api/charging-plan/generate	Multi-agent coordinator charging plan generation
POST	/api/support/analyze	Validation & Support Agent ticket triage, categorization, and refund recommendation (secured via X-Agent-Service-Key)
POST	/api/workflows/support	Coordinator Agent support triage workflow execution (secured via X-Agent-Service-Key)

8.2 Human-in-the-Loop Approval Triggers

To prevent autonomous AI agents from executing unapproved monetary or operational actions, the following conditions halt execution into a PendingApproval state requiring manual confirmation via the React web portal:

Trigger Condition	Threshold / Rule	Assigned Reviewer
Wallet / invoice refund request	AI-triaged support ticket recommends a refund exceeding \$15.00	Customer Support Manager
High-threshold loyalty redemption	Redemption exceeds 5,000 points or \$50.00 equivalent value	Platform Administrator
Waitlist reservation override	AI-generated itinerary bumps a lower-priority waitlisted booking	Station Owner
Meter Discrepancy Audits	Physical staff overrides departing from calculated values by more than 15%	Platform Administrator

9. Appendices

9.1 Summary of Architecture Decision Records

ADR	Title	Summary & Architectural Justification
ADR-01	State Management Frameworks	Riverpod (Flutter) and TanStack Query / React Query (React) selected for predictable, testable client-side state.
ADR-02	Agentic AI Framework & Orchestration	LangGraph selected for structured, graph-based multi-agent orchestration with deterministic state transitions.
ADR-03	Concurrency Control for Slot Allocation	PostgreSQL exclusion constraints combined with pessimistic row-locking (SELECT ... FOR UPDATE) chosen over optimistic concurrency to guarantee zero double-bookings across online reservations and on-site walk-ins.
ADR-04 (Revised)	Hybrid Session Telemetry	Replaces pure mathematical calculation with a dual-layer strategy. Baseline energy is mathematically computed from charger output and duration; staff provides manual overrides from physical charger screens to reflect battery tapering without requiring live IoT telemetry.
ADR-05 (Revised)	Internal Ledger & POS Cash Management	Eliminates external payment gateway dependencies. Advance bookings utilize an internal virtual wallet ledger, while on-site walk-ins and post-charge balances support physical cash collection recorded directly by staff.
ADR-06	Cloudinary for Meter Photo Evidence Storage	To substantiate physical meter overrides without burdening PostgreSQL with binary BLOB storage, the mobile client uploads physical meter display photos directly to Cloudinary using signed parameters. The returned secure HTTPS URL is persisted in ChargingSessions.MeterPhotoUrl and served through Cloudinary CDN to web and mobile audit screens.

9.2 Revision History

Version	Date	Description & Primary Author
1.0	17 August 2026	Initial baseline SRS covering all 4 core functions, baseline 3NF schema, interface specs, non-functional requirements, and Agentic AI subsystem.
2.0	20 September 2026	Comprehensive revision to Hybrid Operational Model: added on-site staff POS workflow, walk-in reservations with null DriverId, manual physical meter kWh overrides, Cloudinary photographic proof capture (ADR-06), cash settlement logging (ADR-05 revised), expanded platform admin vehicle oversight, and CI/CD test automation metrics.